

Practical No 3

Aim: Implement algorithms to generate and verify message authentication codes (MACs) for ensuring data integrity and authenticity.

A) CLI Method:

Code:

```
import hmac
import hashlib
```

```
def generate_mac(message: str, key: str) -> str:
    """Generates an HMAC-SHA256 tag for the given message and key."""
    message_bytes = message.encode('utf-8')
    key_bytes = key.encode('utf-8')

    mac_obj = hmac.new(key_bytes, message_bytes, hashlib.sha256)
    return mac_obj.hexdigest()

def verify_mac(received_message: str, key: str, received_mac: str) -> bool:
    """Verifies if the received MAC matches the MAC generated from the received message."""
    expected_mac = generate_mac(received_message, key)

    return hmac.compare_digest(expected_mac, received_mac)

if __name__ == '__main__':
    print("=== HMAC-SHA256 Generator & Verifier ===")

    while True:
        print("\nOptions:")
        print("1. Generate MAC (Sender)")
        print("2. Verify MAC (Receiver)")
        print("3. Exit")

        choice = input("Select an option (1/2/3): ").strip()

        if choice == '1':
            msg = input("Enter the message: ")
            secret = input("Enter the shared secret key: ")

            if msg and secret:
                mac_tag = generate_mac(msg, secret)
                print("\n[+] MAC GENERATED SUCCESSFULLY")
                print(f"--> MAC Tag: {mac_tag}")
            else:
                print("Error: Message or secret key is empty.")
        elif choice == '2':
            received_msg = input("Enter the received message: ")
            received_mac = input("Enter the received MAC: ")
            key = input("Enter the shared secret key: ")

            if received_msg and received_mac and key:
                is_valid = verify_mac(received_msg, key, received_mac)
                if is_valid:
                    print("\n[+] MAC VERIFIED SUCCESSFULLY")
                else:
                    print("\n[-] MAC NOT VERIFIED")
            else:
                print("Error: Received message, MAC, or key is empty.")
        elif choice == '3':
            print("Exiting...")
            break
        else:
            print("Invalid option. Please choose from 1, 2, or 3.")
```

SHETH LUJ AND SIR MV COLLEGE
Cyber & Information Security

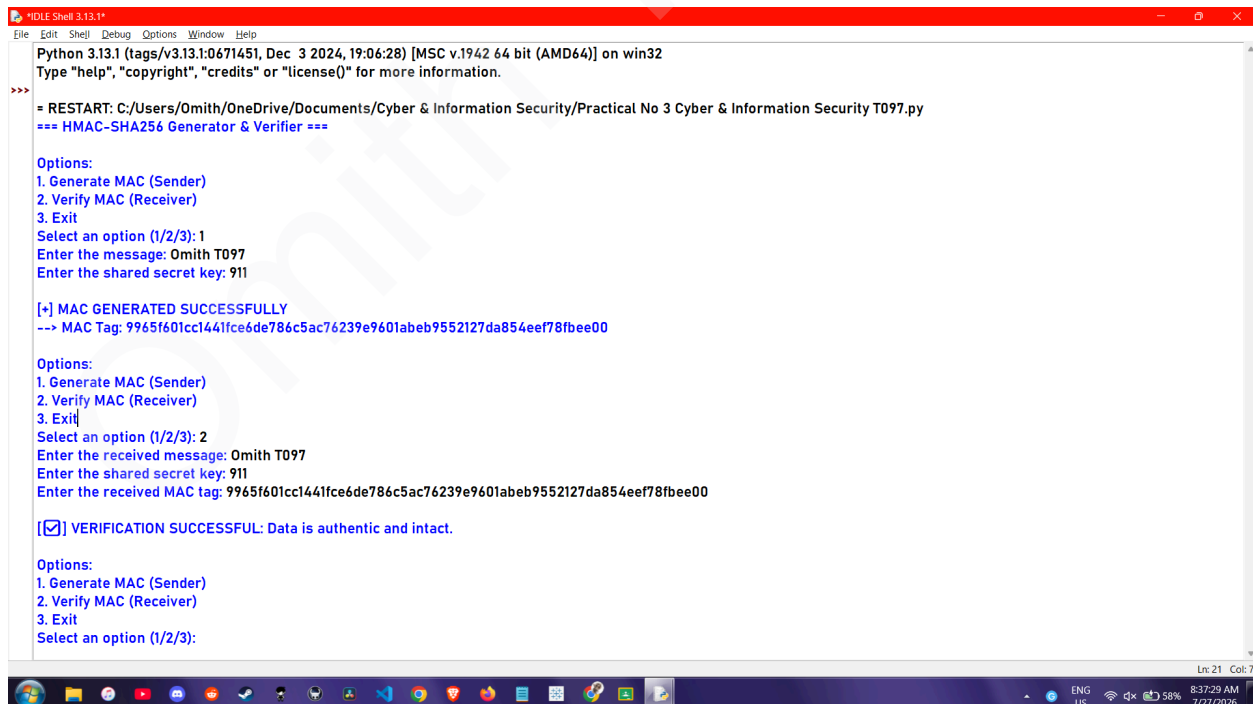
```
print("[-] Message and Secret Key cannot be empty.")

elif choice == '2':
    recv_msg = input("Enter the received message: ")
    secret = input("Enter the shared secret key: ")
    recv_mac = input("Enter the received MAC tag: ").strip()

    if recv_msg and secret and recv_mac:
        is_valid = verify_mac(recv_msg, secret, recv_mac)
        if is_valid:
            print("\n[✓] VERIFICATION SUCCESSFUL: Data is authentic and intact.")
        else:
            print("\n[✗] VERIFICATION FAILED: Data was tampered with or key is incorrect!")
    else:
        print("[-] All fields must be filled for verification.")

elif choice == '3':
    print("Exiting program. Goodbye!")
    break
else:
    print("[-] Invalid option. Please try again.")
```

Output:



The screenshot shows a Python IDE window titled "IDLE Shell 3.13.1". The code being executed is a program for HMAC-SHA256 generation and verification. The output shows the user selecting option 1 (Generate MAC), entering the message "Omith T097" and the shared secret key "911". The program successfully generates a MAC tag: "9965f601cc1441fce6de786c5ac76239e9601abeb9552127da854eef78fbee00". The user then selects option 2 (Verify MAC), enters the same message and secret key, and the program successfully verifies the data as authentic and intact.

```
Python 3.13.1 (tags/v3.13.1:0671451, Dec 3 2024, 19:06:28) [MSC v.1942 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Omith/OneDrive/Documents/Cyber & Information Security/Practical No 3 Cyber & Information Security T097.py
=== HMAC-SHA256 Generator & Verifier ===

Options:
1. Generate MAC (Sender)
2. Verify MAC (Receiver)
3. Exit
Select an option (1/2/3): 1
Enter the message: Omith T097
Enter the shared secret key: 911

[+] MAC GENERATED SUCCESSFULLY
--- MAC Tag: 9965f601cc1441fce6de786c5ac76239e9601abeb9552127da854eef78fbee00

Options:
1. Generate MAC (Sender)
2. Verify MAC (Receiver)
3. Exit
Select an option (1/2/3): 2
Enter the received message: Omith T097
Enter the shared secret key: 911
Enter the received MAC tag: 9965f601cc1441fce6de786c5ac76239e9601abeb9552127da854eef78fbee00

[✓] VERIFICATION SUCCESSFUL: Data is authentic and intact.

Options:
1. Generate MAC (Sender)
2. Verify MAC (Receiver)
3. Exit
Select an option (1/2/3):
```

B) GUI Method:

Code:

```
import flet as ft
```

```
import hmac
```

```
import hashlib
```

```
# --- Cryptographic Logic ---
```

```
def generate_mac(message: str, key: str) -> str:
```

```
    mac_obj = hmac.new(key.encode('utf-8'), message.encode('utf-8'), hashlib.sha256)
```

```
    return mac_obj.hexdigest()
```

```
def verify_mac(message: str, key: str, provided_mac: str) -> bool:
```

```
    expected_mac = generate_mac(message, key)
```

```
    return hmac.compare_digest(expected_mac, provided_mac)
```

```
# --- Flet Modern GUI ---
```

```
def main(page: ft.Page):
```

```
    # Window Settings
```

```
    page.title = "HMAC Authenticator Studio"
```

```
    page.window.width = 600
```

```
    page.window.height = 750
```

```
    page.theme_mode = ft.ThemeMode.DARK
```

```
    page.bgcolor = "#0D1117" # GitHub dark background
```

```
    page.padding = 30
```

```
    page.horizontal_alignment = ft.CrossAxisAlignment.CENTER
```

```
    # Palette
```

```
    CYAN = "#00FFA3"
```

```
    PURPLE = "#A371F7"
```

```
    RED = "#FF7B72"
```

```
    GREEN = "#3FB950"
```

```
    CARD_BG = "#161B22"
```

```
# --- UI Components ---
```

```
header = ft.Row(
```

```
    controls=[
```

```
        ft.Icon(ft.Icons.VERIFIED_USER, color=CYAN, size=32),
```

```
        ft.Text("HMAC Studio", size=28, weight=ft.FontWeight.W_800, color=ft.Colors.WHITE)
```

```
    ],
```

```
    alignment=ft.MainAxisAlignment.CENTER
```

```
)
```

```
# Input Fields
```

SHETH LUJ AND SIR MV COLLEGE
Cyber & Information Security

```
message_input = ft.TextField(  
    label="Message Payload",  
    multiline=True,  
    min_lines=2,  
    max_lines=3,  
    border_color="#30363D",  
    focused_border_color=CYAN,  
    border_radius=10  
)  
  
key_input = ft.TextField(  
    label="Shared Secret Key",  
    password=True,  
    can_reveal_password=True,  
    border_color="#30363D",  
    focused_border_color=PURPLE,  
    border_radius=10  
)  
  
mac_input = ft.TextField(  
    label="MAC Tag (Paste here for verification)",  
    border_color="#30363D",  
    focused_border_color=RED,  
    text_style=ft.TextStyle(font_family="monospace", size=12),  
    border_radius=10  
)  
  
terminal_output = ft.TextField(  
    label="System Terminal",  
    multiline=True,  
    min_lines=4,  
    max_lines=6,  
    read_only=True,  
    border_color="#30363D",  
    focused_border_color=CYAN,  
    text_style=ft.TextStyle(font_family="monospace"),  
    border_radius=10  
)  
  
# Input Container (Card)  
input_card = ft.Container(  
    content=ft.Column([  
        ft.Text("DATA INPUT", weight=ft.FontWeight.BOLD, color="#8B949E"),  
        message_input,  
    ])
```

SHETH LUJ AND SIR MV COLLEGE
Cyber & Information Security

```
        key_input,
        mac_input
    ], spacing=15),
    bgcolor=CARD_BG,
    padding=20,
    border_radius=15,
    width=float("inf")
)

# --- Handlers ---
def show_toast(msg, color):
    page.overlay.append(ft.SnackBar(ft.Text(msg, weight=ft.FontWeight.BOLD), bgcolor=color,
open=True))
    page.update()

def handle_generate(e):
    if not message_input.value or not key_input.value:
        show_toast("⚠ Enter both Message and Secret Key to generate a MAC.", RED)
        return

    mac_tag = generate_mac(message_input.value, key_input.value)
    mac_input.value = mac_tag # Auto-fill the MAC input for easy copying
    terminal_output.value = f"[+] MAC GENERATED SUCCESSFULLY:\n{mac_tag}"
    show_toast("🔒 MAC Generated", GREEN)
    page.update()

def handle_verify(e):
    if not message_input.value or not key_input.value or not mac_input.value:
        show_toast("⚠ Message, Key, and MAC Tag are required for verification.", RED)
        return

    is_valid = verify_mac(message_input.value, key_input.value, mac_input.value.strip())
    if is_valid:
        terminal_output.value = "[✅] INTEGRITY VERIFIED:\nThe message is authentic and
has not been tampered with."
        show_toast("✅ Verification Passed", GREEN)
    else:
        terminal_output.value = "[❌] VERIFICATION FAILED:\nWARNING: Data tampering
detected OR incorrect secret key!"
        show_toast("❌ Verification Failed", RED)
        page.update()

# --- Buttons ---
# Moved colors directly into the ButtonStyle to comply with the new ft.Button API
```

SHETH LUJ AND SIR MV COLLEGE
Cyber & Information Security

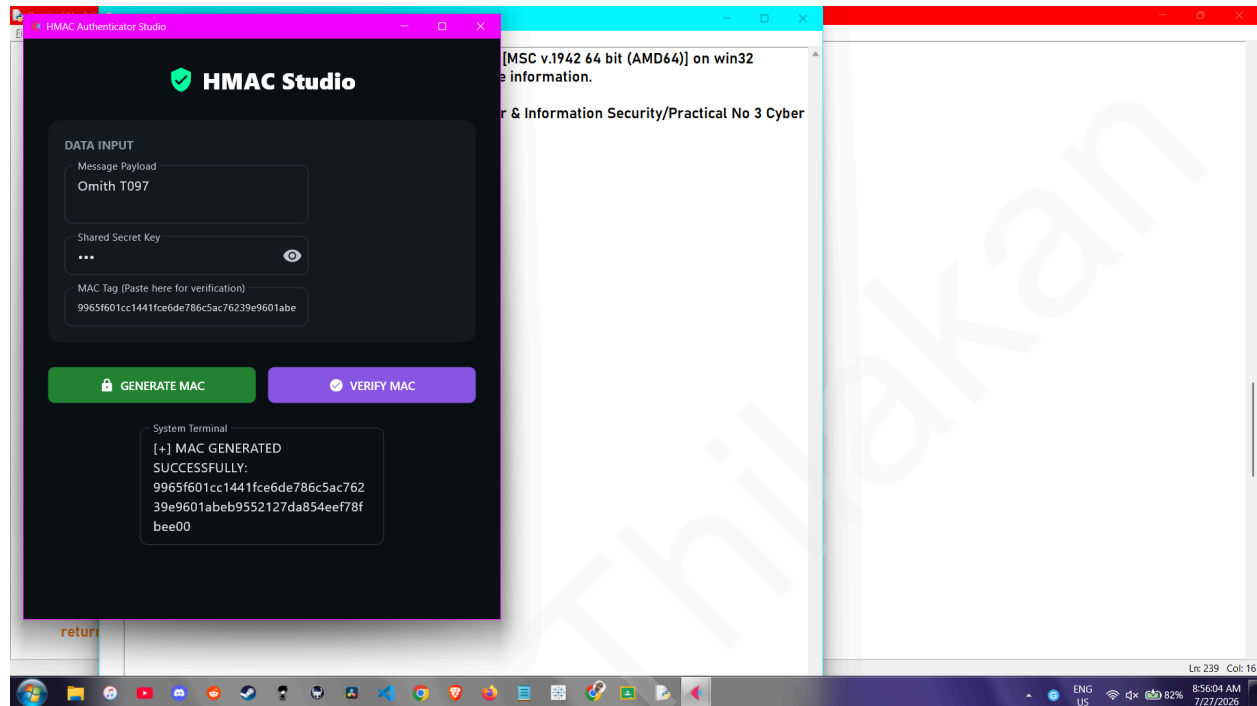
```
btn_style_gen = ft.ButtonStyle(  
    shape=ft.RoundedRectangleBorder(radius=8),  
    padding=20,  
    bgcolor="#238636",  
    color=ft.Colors.WHITE  
)  
  
btn_style_ver = ft.ButtonStyle(  
    shape=ft.RoundedRectangleBorder(radius=8),  
    padding=20,  
    bgcolor="#8957E5",  
    color=ft.Colors.WHITE  
)  
  
btn_row = ft.Row(  
    controls=[  
        ft.Button(  
            "GENERATE MAC",  
            icon=ft.Icons.LOCK,  
            on_click=handle_generate,  
            style=btn_style_gen,  
            expand=True  
        ),  
        ft.Button(  
            "VERIFY MAC",  
            icon=ft.Icons.CHECK_CIRCLE,  
            on_click=handle_verify,  
            style=btn_style_ver,  
            expand=True  
        ),  
    ],  
    spacing=15  
)  
  
# Render  
page.add(  
    header,  
    ft.Divider(height=10, color="transparent"),  
    input_card,  
    ft.Divider(height=10, color="transparent"),  
    btn_row,  
    ft.Divider(height=10, color="transparent"),  
    terminal_output  
)
```

SHETH LUJ AND SIR MV COLLEGE
Cyber & Information Security

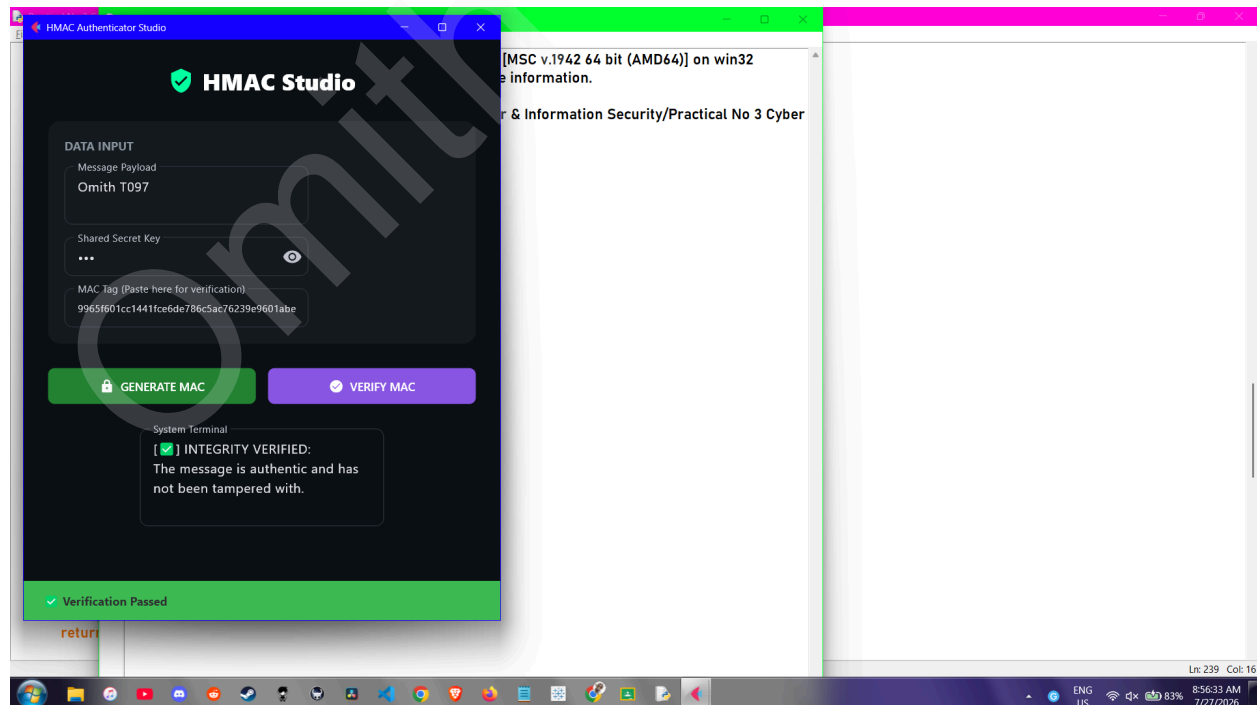
```
if __name__ == "__main__":  
    ft.run(main)
```

Output:

Generating MAC Code:



Verifying MAC Code:



Omith Thilakan
T097